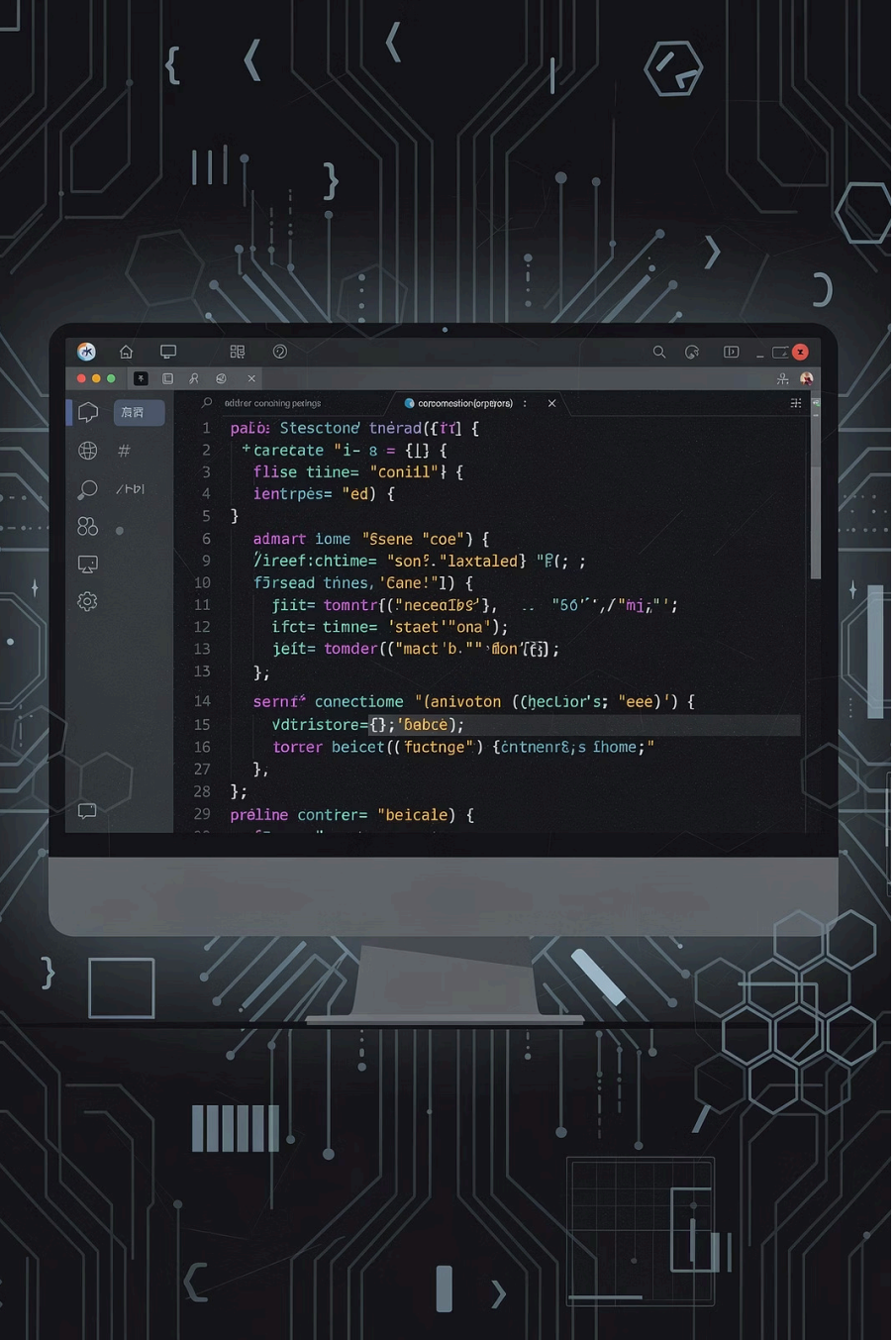


Programación Orientada a Objetos en Python

Módulos 8, 9 y 10 · 5 horas de aprendizaje gradual

MSC MANUEL MALDONADO

DGTIC · UNAM

Contenido del curso

¿Qué aprenderemos?

Este material cubre los fundamentos de la Programación Orientada a Objetos (POO) en Python, diseñado para estudiantes sin experiencia previa. Avanzaremos de forma gradual, con ejemplos claros y ejercicios prácticos.

01

Módulo 8 · 1 hora

Elementos básicos de la POO: concepto de objeto, anatomía y beneficios.

02

Módulo 9 · 2 horas

Clases y objetos: atributos, métodos, constructores y destructores.

03

Módulo 10 · 2 horas

Encapsulamiento: modularidad, ocultamiento y protección de variables.

MÓDULO 8

¿Qué es un Objeto?

Un **objeto** es una entidad que combina **datos (atributos)** y **comportamiento (métodos)** en una sola unidad. Representa algo del mundo real o conceptual.

Atributos (datos)

- color: "rojo"
- marca: "Toyota"
- velocidad: 50
- número de puertas: 4

Métodos (comportamiento)

- acelerar()
- frenar()
- encender()
- apagar()

MÓDULO 8

Anatomía de un Objeto

Todo objeto en POO tiene tres componentes fundamentales que lo definen y distinguen de los demás. Entenderlos es clave para comprender cómo funciona la orientación a objetos.

🪪 Identidad

Referencia única que permite distinguir un objeto de otro. En Python, cada objeto tiene su propia dirección en memoria, como el número de serie de un coche.

📋 Estado

Conjunto de valores de sus atributos en un momento dado. Por ejemplo: `color="rojo"`, `velocidad=50`. El estado puede cambiar con el tiempo.

⚙️ Comportamiento

Operaciones que el objeto puede realizar. Se implementan como métodos: `acelerar()`, `frenar()`. Definen qué puede hacer el objeto.

MÓDULO 8

Beneficios de la POO

La programación orientada a objetos ofrece ventajas prácticas que hacen el código más robusto, mantenible y reutilizable en proyectos de cualquier escala.

Reutilización

Las clases se heredan y reutilizan en distintos proyectos, reduciendo el código duplicado y el tiempo de desarrollo.

Modularidad

Cada clase es una unidad independiente. Facilita el trabajo en equipo y el mantenimiento a largo plazo.

Abstracción

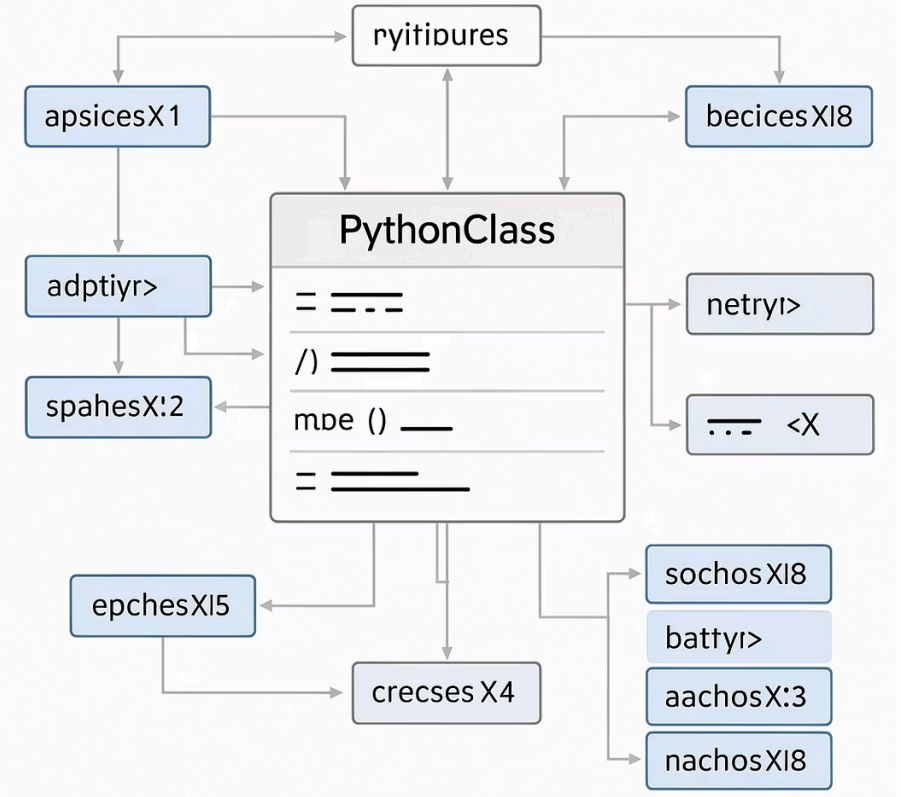
Se ocultan los detalles complejos y se expone solo lo necesario. El usuario no necesita entender la implementación interna.

Encapsulamiento

Los datos internos están protegidos contra accesos no autorizados, proporcionando mayor seguridad y control.

Polimorfismo

Una misma interfaz puede comportarse de diferentes formas, haciendo el código más flexible y extensible.



MÓDULO 9

¿Qué es una Clase?

Una **clase** es una **plantilla o molde** que define la estructura y el comportamiento que tendrán todos los objetos creados a partir de ella. La clase es el plano; el objeto es la construcción.

```
class Coche:
    """Clase que representa un coche"""
    pass

# Crear objetos (instancias) de la clase
mi_coche = Coche()
tu_coche = Coche()

print(type(mi_coche))  # <class '__main__.Coche'>
```

💡 **Analogía clave:** La clase `Coche` es como el molde de una fábrica. Cada coche que sale del molde es un objeto distinto, con su propio color y velocidad, pero todos comparten la misma estructura.

MÓDULO 9

Atributos: de Clase vs. de Instancia

En Python existen dos tipos de atributos. Entender la diferencia es fundamental para diseñar clases correctamente.

Atributos de Clase

Compartidos por **todas** las instancias. Se definen directamente en la clase, fuera del constructor.

```
class Coche:
    cantidad_ruedas = 4
```

Acceso: `Coche.cantidad_ruedas`

Atributos de Instancia

Únicos para **cada objeto**. Se definen dentro del método `__init__` usando `self`.

```
def __init__(self, color):
    self.color = color
```

Acceso: `coche1.color`

```
coche1 = Coche("rojo", "Toyota")
coche2 = Coche("azul", "Honda")
print(coche1.color) # rojo
print(Coche.cantidad_ruedas) # 4
```

MÓDULO 9

Métodos: El Comportamiento de los Objetos

Los **métodos** son funciones definidas dentro de una clase. Siempre reciben `self` como primer parámetro, que representa al propio objeto que llama al método.

```
class Coche:
 def __init__(self, color, marca):
 self.color = color
 self.marca = marca
 self.velocidad = 0

 def acelerar(self, incremento):
 self.velocidad += incremento
 print(f"El coche {self.marca} va a {self.velocidad} km/h")

 def frenar(self, decremento):
 self.velocidad -= decremento
 if self.velocidad < 0:
 self.velocidad = 0
 print(f"Velocidad actual: {self.velocidad} km/h")

# Uso
mi_coche = Coche("rojo", "Toyota")
mi_coche.acelerar(20) # El coche Toyota va a 20 km/h
mi_coche.frenar(5) # Velocidad actual: 15 km/h
```

MÓDULO 9

El Constructor `__init__`

El **constructor** es un método especial que se ejecuta **automáticamente** cada vez que se crea un nuevo objeto. En Python se denomina `__init__` y sirve para inicializar los atributos del objeto.

```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad
        print(f"Se ha creado a {self.nombre}")

persona1 = Persona("Ana", 25)   # "Se ha creado a Ana"
persona2 = Persona("Luis", 30)  # "Se ha creado a Luis"
```

Métodos especiales más comunes en Python

Método	Se ejecuta cuando...
`__init__(self)`	Se crea un objeto (constructor)
`__del__(self)`	El objeto es destruido (destructor)
`__str__(self)`	Se usa `print(objeto)` o `str(objeto)`
`__repr__(self)`	Representación oficial del objeto

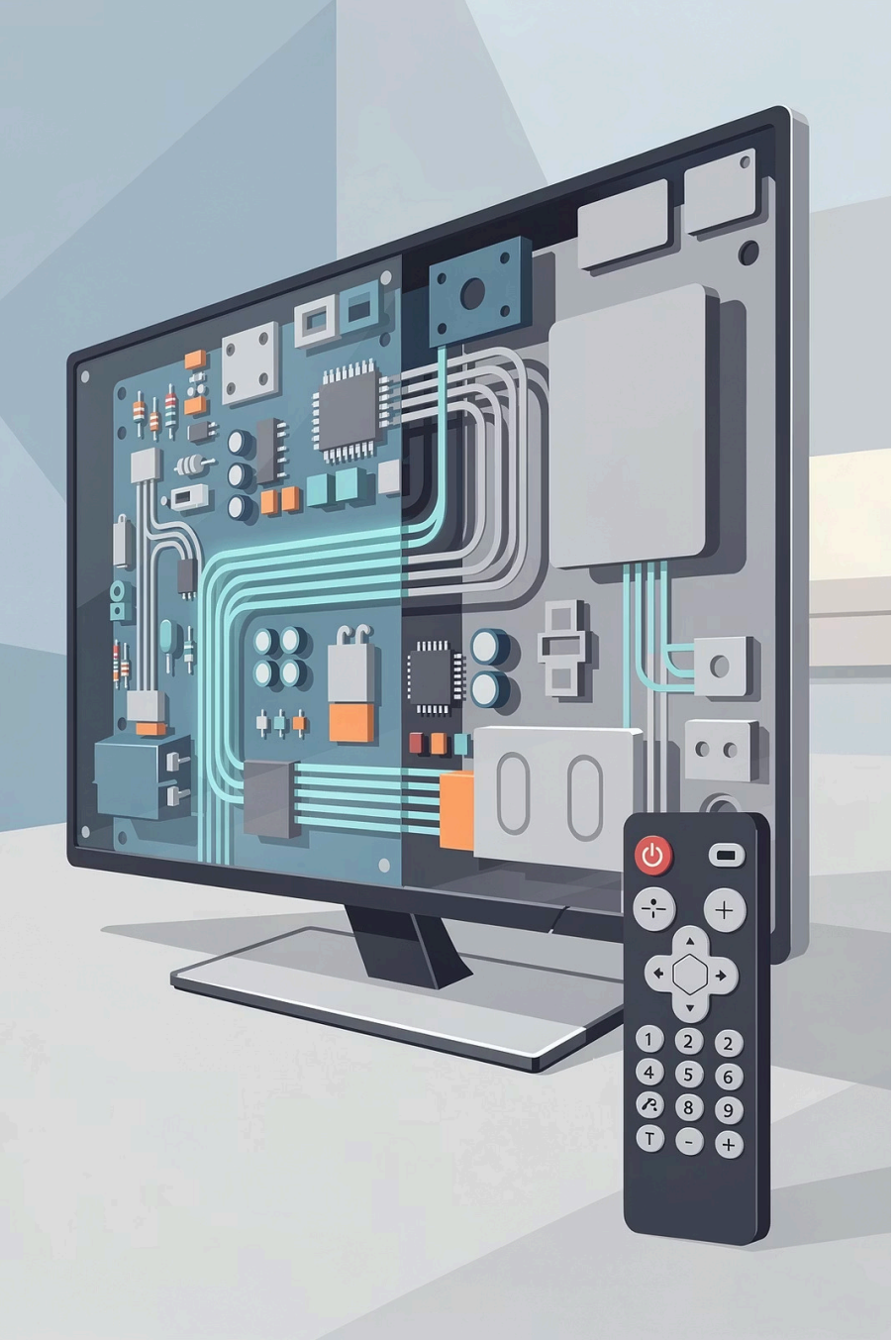
MÓDULO 9

Destructores: `__del__`

El **destructor** `__del__` se ejecuta automáticamente cuando un objeto es eliminado de la memoria, ya sea por el recolector de basura de Python o de forma explícita con `del`. En Python no es tan habitual usarlo porque la gestión de memoria es automática, pero es útil para cerrar recursos como archivos o conexiones.

```
class Archivo:
    def __init__(self, nombre):
        self.nombre = nombre
        print(f"Archivo {nombre} abierto")

    def __del__(self):
        print(f"Archivo {self.nombre} cerrado (destructor)")

# Crear objeto
archivo = Archivo("datos.txt")
# Output: Archivo datos.txt abierto

# Eliminar explícitamente
del archivo
# Output: Archivo datos.txt cerrado (destructor)
```

⚠️ **Nota importante:** Aunque `__del__` existe, para liberar recursos en Python se recomienda usar el gestor de contexto `with`, que garantiza la liberación del recurso sin depender del recolector de basura.



MÓDULO 10

Encapsulamiento y Modularidad

El **encapsulamiento** es el principio de ocultar los detalles internos de una clase y exponer solo una interfaz clara y controlada al exterior. Así como un televisor muestra botones al usuario pero esconde sus circuitos internos, nuestras clases deben proteger su implementación interna.

📦 Modularidad

Capacidad de dividir el programa en clases independientes que pueden desarrollarse y probarse por separado. Facilita el trabajo en equipo y aísla fallos.

🔒 Ocultamiento

Los detalles internos de la clase se mantienen ocultos. Solo se expone lo necesario a través de métodos públicos bien definidos.

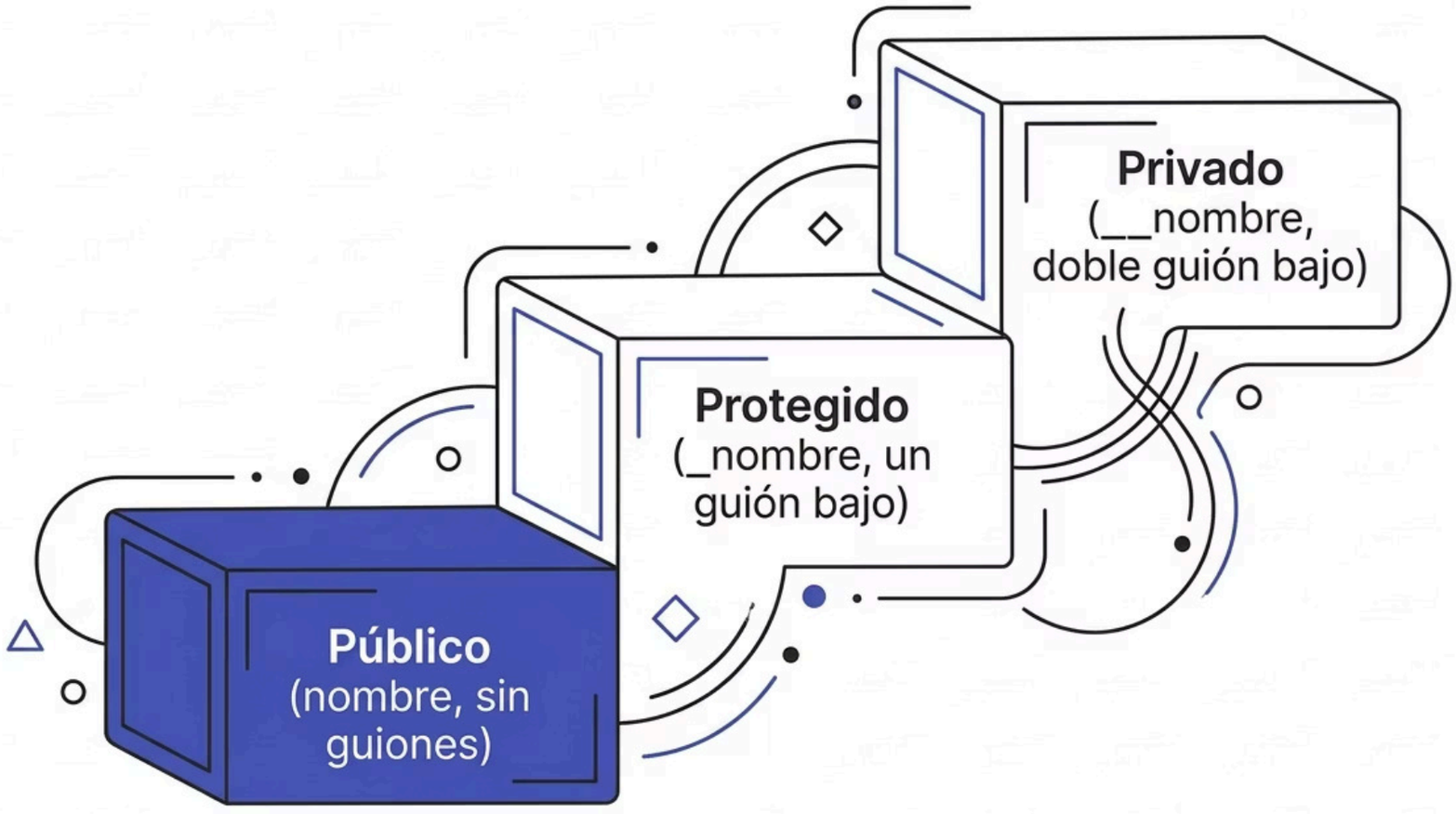
🛡️ Protección

Evita que código externo modifique directamente los datos internos, previniendo errores y comportamientos inesperados en el sistema.

MÓDULO 10

Niveles de Acceso en Python

Python usa **convenciones de nomenclatura** para indicar el nivel de acceso de atributos y métodos, a diferencia de Java o C# que tienen palabras clave como `private`.



La escalera representa el nivel creciente de restricción: público (base amplia y accesible), protegido (uso interno recomendado) y privado (acceso bloqueado mediante name mangling).

	Variable	Example	Access
Public Access Public library	name	<code name= { = 'n;' ... </>	
Protected single underscore Semi-private key Restricted group	_name	<code name= { = '.; ... ... </>	
Private double underscore Single individual Fortified server room	__name	Name mangling <code name> </> (___)	STOP
Triple underscore super-private Biometric scans Top-secret clearance	___name	<ode> access { '!> ... </>	

MÓDULO 10

Miembros Privados: `__atributo`

El **doble guion bajo** al inicio del nombre activa el *name mangling*: Python renombra internamente el atributo para dificultar su acceso desde fuera de la clase.

```
class CuentaBancaria:
 def __init__(self, titular, saldo):
 self.titular = titular
 self.__saldo = saldo # Atributo privado

 def depositar(self, monto):
 if monto > 0:
 self.__saldo += monto

 def obtener_saldo(self):
 return self.__saldo

# Uso
cuenta = CuentaBancaria("Ana", 1000)
print(cuenta.titular) #
```

MÓDULO 10

Miembros Protegidos: `_atributo`

El **guion bajo simple** es una convención que indica "este miembro es de uso interno". Python no impide el acceso, pero comunica a otros desarrolladores que no deben usarlo fuera de la clase o sus subclases.

```
class Vehiculo:
 def __init__(self, marca):
 self._marca = marca # Protegido (convención)

 def _metodo_interno(self): # Método protegido
 print("Lógica interna")

class Coche(Vehiculo):
 def mostrar_marca(self):
 print(f"Marca: {self._marca}") #
```

EJERCICIOS PRÁCTICOS

Manos a la Obra: 4 Ejercicios

Estos ejercicios están diseñados para consolidar los conceptos de los tres módulos. Se recomienda resolverlos en orden, ya que cada uno se apoya en el anterior.

1

Clase básica (30 min)

Define una clase `Estudiante` con atributos `nombre`, `edad`, `carrera` y un método `presentarse()`. Crea 3 estudiantes y haz que se presenten.

2

Encapsulamiento (30 min)

Clase `CuentaBancaria` con atributo privado `__saldo`, métodos `depositar()`, `retirar()`, `consultar_saldo()` y un método protegido `_registrar_movimiento()`.

3

Clase vs. instancia (20 min)

Clase `Producto` con atributo de clase `contador` que se incrementa con cada nuevo producto creado. Muestra el total después de crear varios productos.

4

Constructor y destructor (20 min)

Clase `ConexionBD` que imprime "Conexión establecida" en `__init__` y "Conexión cerrada" en `__del__`. Crea el objeto y luego elimínalo con `del`.

EJERCICIO 2 — SOLUCIÓN GUIADA

CuentaBancaria con Encapsulamiento

Este ejercicio integra todos los niveles de protección vistos en el Módulo 10. Observa cómo los métodos públicos actúan como guardianes de los datos privados.

```
class CuentaBancaria:
 def __init__(self, titular, saldo):
 self.titular = titular # Público
 self.__saldo = saldo # Privado

 def depositar(self, monto):
 if monto > 0:
 self.__saldo += monto
 self._registrar_movimiento("depósito", monto)

 def retirar(self, monto):
 if monto > self.__saldo:
 print("
```

EJERCICIO 3 — SOLUCIÓN GUIADA

Atributos de Clase: Contador de Productos

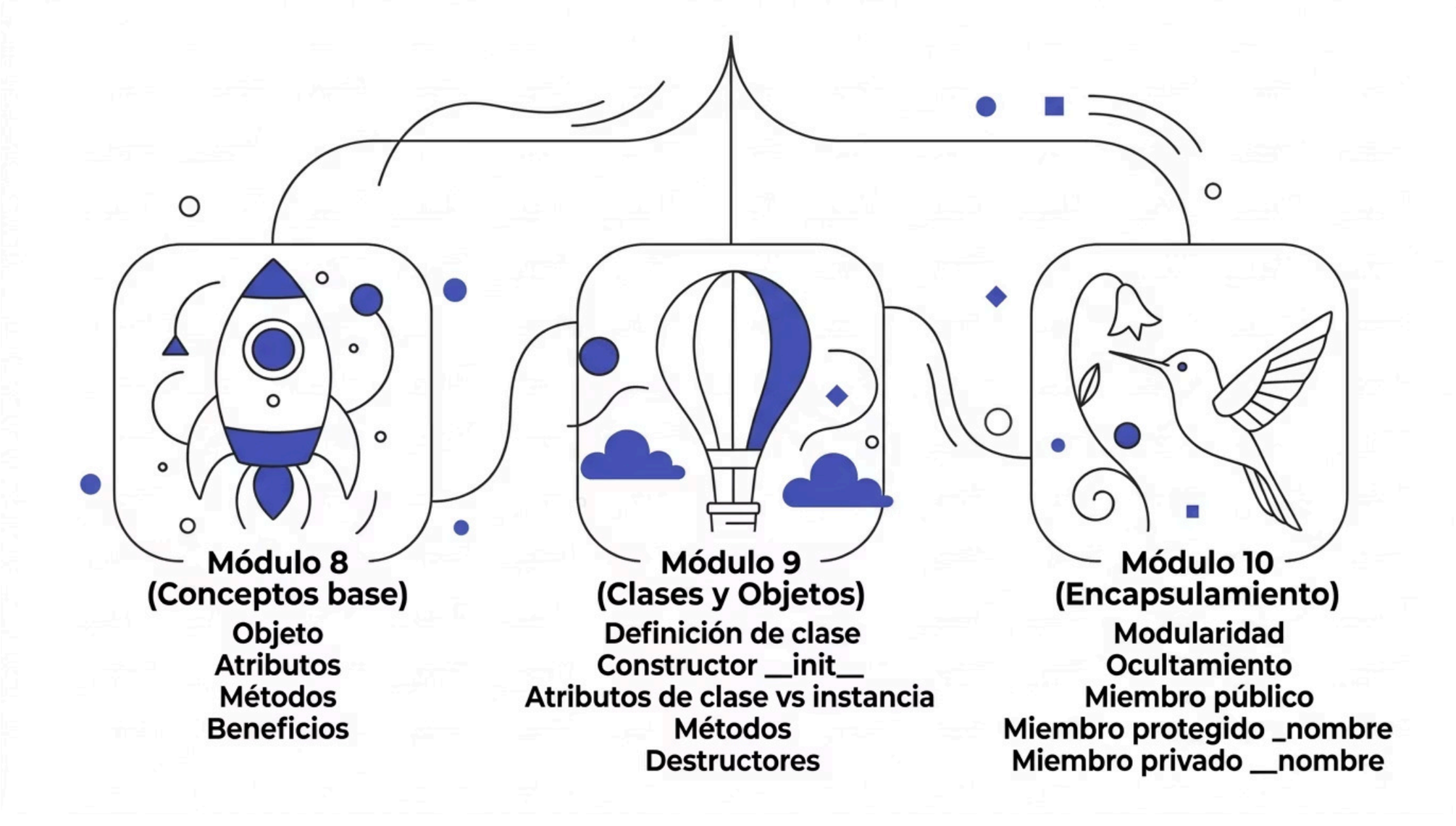
Este ejercicio demuestra cuándo usar un atributo de clase: cuando el valor debe ser compartido y acumulado por **todas** las instancias, no solo por una.

```
class Producto:
    contador = 0  # Atributo de clase, compartido por todos

    def __init__(self, nombre, precio):
        self.nombre = nombre        # Atributo de instancia
        self.precio = precio        # Atributo de instancia
        Producto.contador += 1      # Incrementar el contador global

    def mostrar(self):
        print(f"{self.nombre}: {self.precio}€")

# Crear productos
p1 = Producto("Manzana", 0.50)
p2 = Producto("Libro Python", 29.99)
p3 = Producto("Teclado", 45.00)

print(f"Productos creados: {Producto.contador}")  # 3
p1.mostrar()  # Manzana: 0.5€
```

💡 **Clave:** Usamos `Producto.contador` (con el nombre de la clase) para modificar el atributo de clase desde dentro del constructor, evitando confusiones con `self.contador`.

Resumen Visual: Los Tres Módulos

Un mapa conceptual de todo lo aprendido en estas 5 horas de POO en Python.



La POO organiza el código en torno a objetos que encapsulan datos y comportamiento, haciendo los programas más modulares, seguros y fáciles de mantener.

Conceptos Clave para Recordar

Clase → Objeto

La clase es el molde. El objeto es la instancia creada a partir de ese molde con sus propios valores.

`self`

Referencia al propio objeto dentro de los métodos. Siempre es el primer parámetro de cualquier método de instancia.

`__init__`

Constructor que se ejecuta automáticamente al crear un objeto. Inicializa los atributos de instancia.

`__` vs `_`

Doble guion = privado (name mangling). Guion simple = protegido (convención). Sin guion = público.

Créditos

Material didáctico desarrollado para el aprendizaje gradual de la Programación Orientada a Objetos en Python, dirigido a estudiantes sin experiencia previa en POO.

Autor

MsC Manuel Maldonado
Diseño instruccional y contenido técnico

Institución

DGTIC · Universidad Nacional Autónoma de México (UNAM)
Dirección General de Cómputo y de Tecnologías de Información y Comunicación

Módulos

Módulo 8, 9 y 10 · Programación Orientada a Objetos en Python · 5 horas de contenido gradual y práctico

MSC MANUEL MALDONADO DGTIC · UNAM